

Lane-Keeping Autonomous Driving

Lane-Keeping Autonomous Driving

1. Course Content
2. Run Example Program
3. Principle Analysis
 - 3.1 Straight Driving Maintenance
 - 3.2 Left and Right Lane Keeping
 - 3.3 Curve Turning
4. Source Code Analysis

1. Course Content

Basic: Start the autonomous driving program and use lane-keeping autonomous driving functionality in the sandbox map

Advanced: Understand the implementation principles of lane-keeping

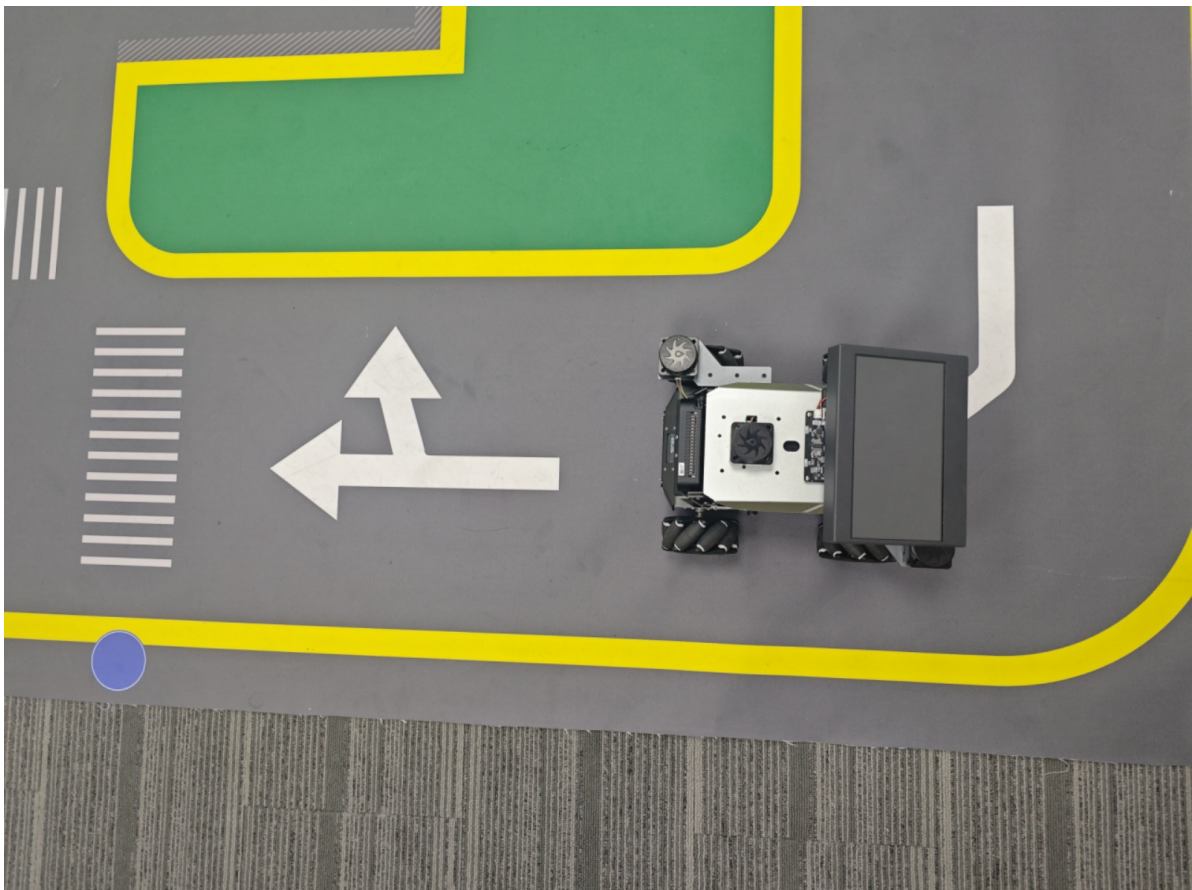
2. Run Example Program

- Use the following command to start microROS agent to connect to the chassis

If the agent has already been started, there is no need to start it repeatedly

```
start_agent
```

- Place the vehicle in the middle position of the lane in the sandbox map



- Start lane-keeping autonomous driving

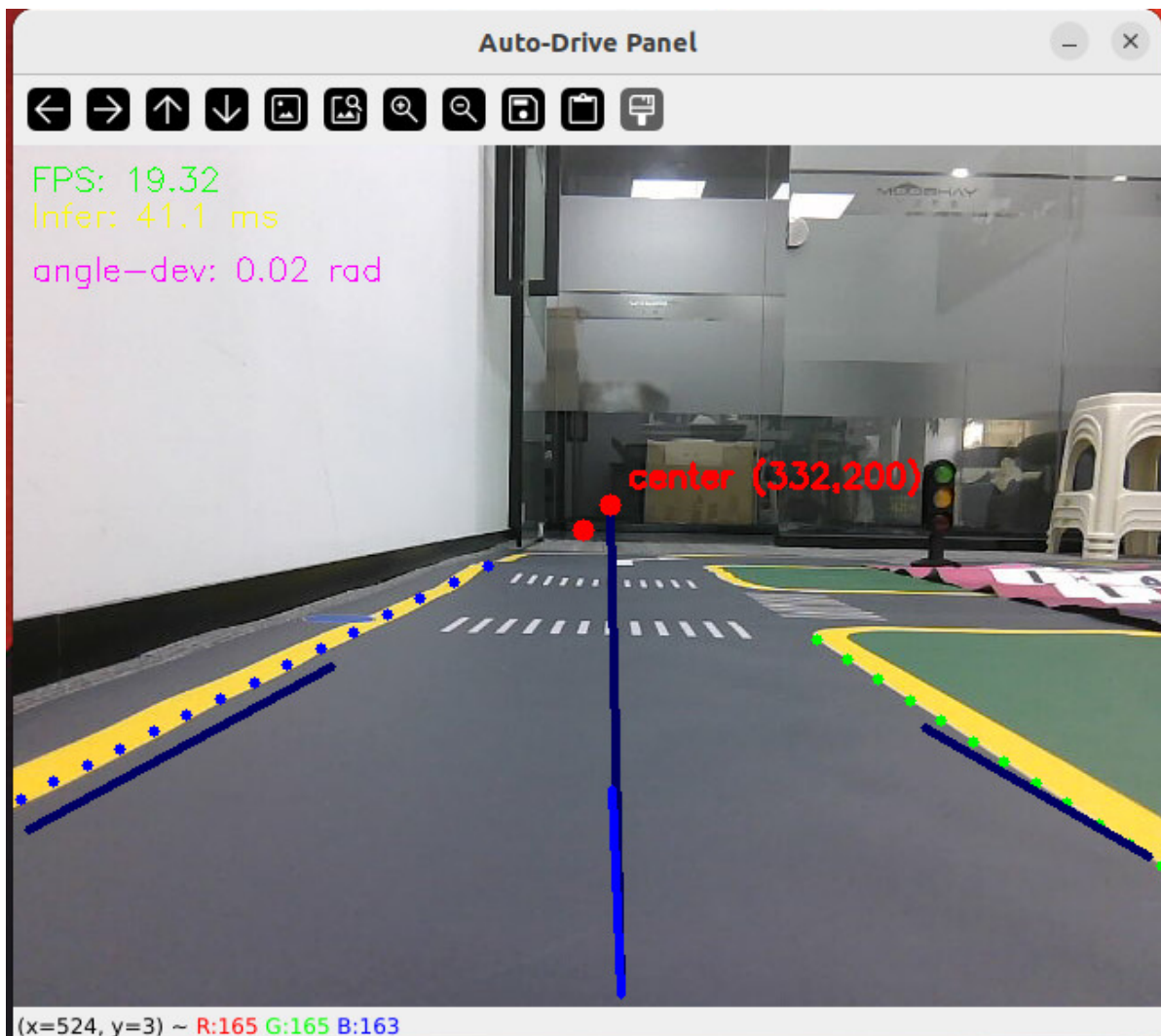
```
ros2 launch m3_bringup auto_drive.launch.py unsigned=True
```

```
jetson@yahboom: ~
jetson@yahboom: ~ 123x17
jetson@yahboom:~$ ros2 launch m3_bringup auto_drive.launch.py unsigned=True
[INFO] [launch]: All log files can be found below /home/jetson/.ros/log/2025-12-03-18-23-42-898157-yahboom-611234
[INFO] [launch]: Default logging verbosity is set to INFO
[INFO] [component_container-1]: process started with pid [611276]
[component_container-1] [INFO] [1764757423.418201055] [camera.camera_container]: Load Library: /home/jetson/yahboomM3_ws/co
de_ws/install/orbbec_camera/lib/liborbbec_camera.so
[component_container-1] [INFO] [1764757423.619091460] [camera.camera_container]: Found class: rclcpp_components::NodeFactor
yTemplate<orbbec_camera::OBCameraNodeDriver>
[component_container-1] [INFO] [1764757423.620925371] [camera.camera_container]: Instantiate class: rclcpp_components::Node
FactoryTemplate<orbbec_camera::OBCameraNodeDriver>
[INFO] [launch_ros.actions.load_composable_nodes]: Loaded node '/camera/camera' in container '/camera/camera_container'
[component_container-1] [INFO] [1764757423.815791498] [camera.camera]: Connecting to the default device
[component_container-1] [INFO] [1764757424.043714594] [camera.camera]: Select device cost 228 ms
[component_container-1] [INFO] [1764757424.043916648] [camera.camera]: Try to connect device via USB2.0
[component_container-1] [INFO] [1764757424.044161552] [camera.camera]: OBCameraNode: use_intra_process: OFF
[component_container-1] [INFO] [1764757424.050397869] [camera.camera]: current time domain: system
[component_container-1] [INFO] [1764757424.115127479] [camera.camera]: Create align filter
[component_container-1] [INFO] [1764757424.116034240] [camera.camera]: Getting LPR to GPU
```

[!NOTE]

Display parameters

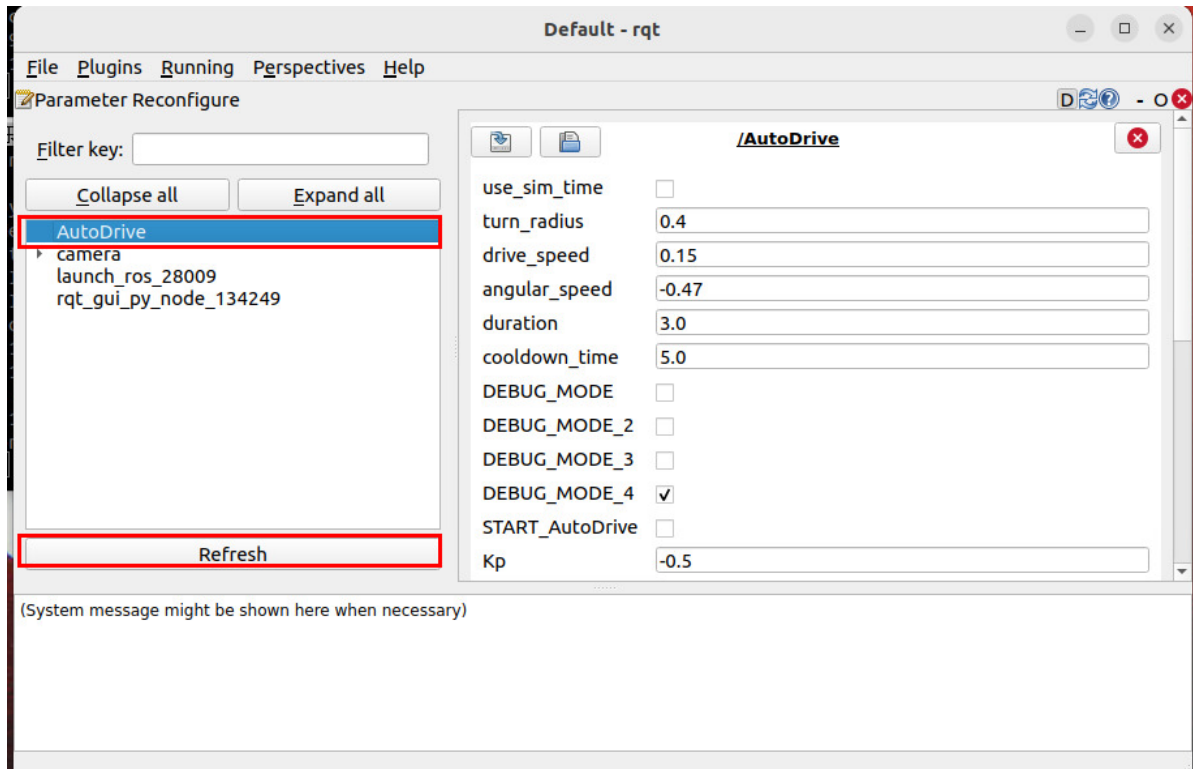
- FPS: Real-time frame rate
- infer: Total inference and computation time per frame
- angle-dev: The angle between the **vehicle body centerline** (blue in the figure) and the **line connecting the lane line intersection point with the centerline starting point**



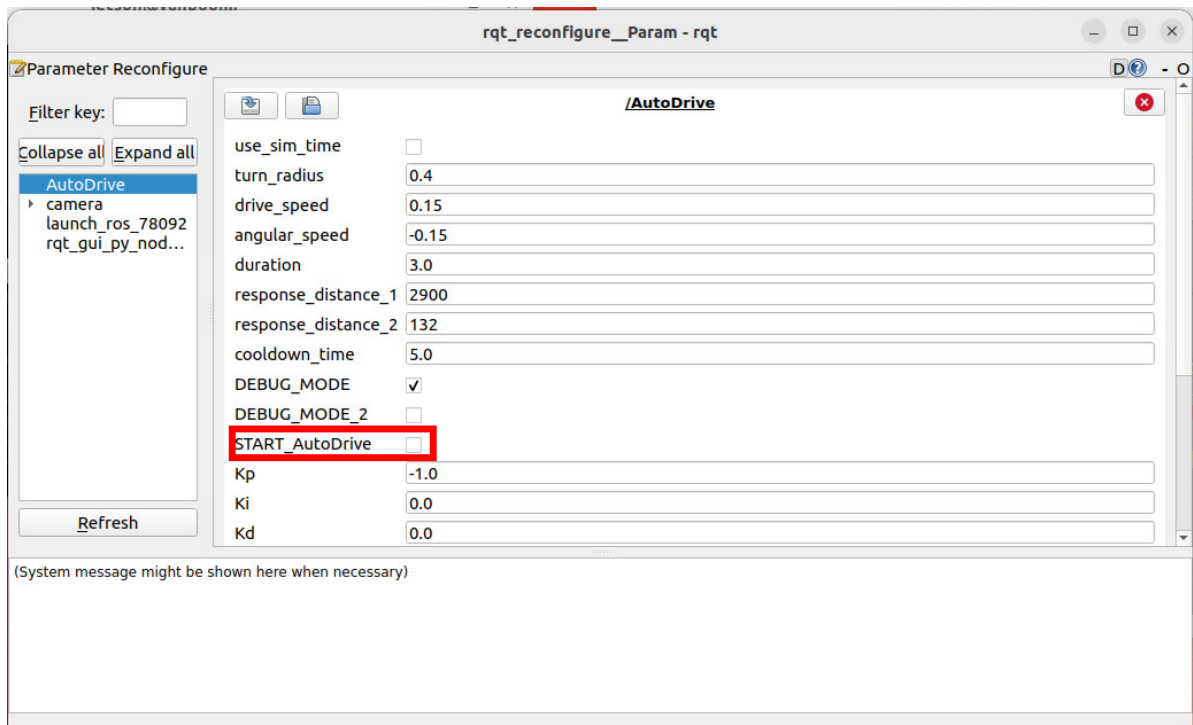
- Start dynamic parameter configuration panel

```
ros2 run rqt_reconfigure rqt_reconfigure
```

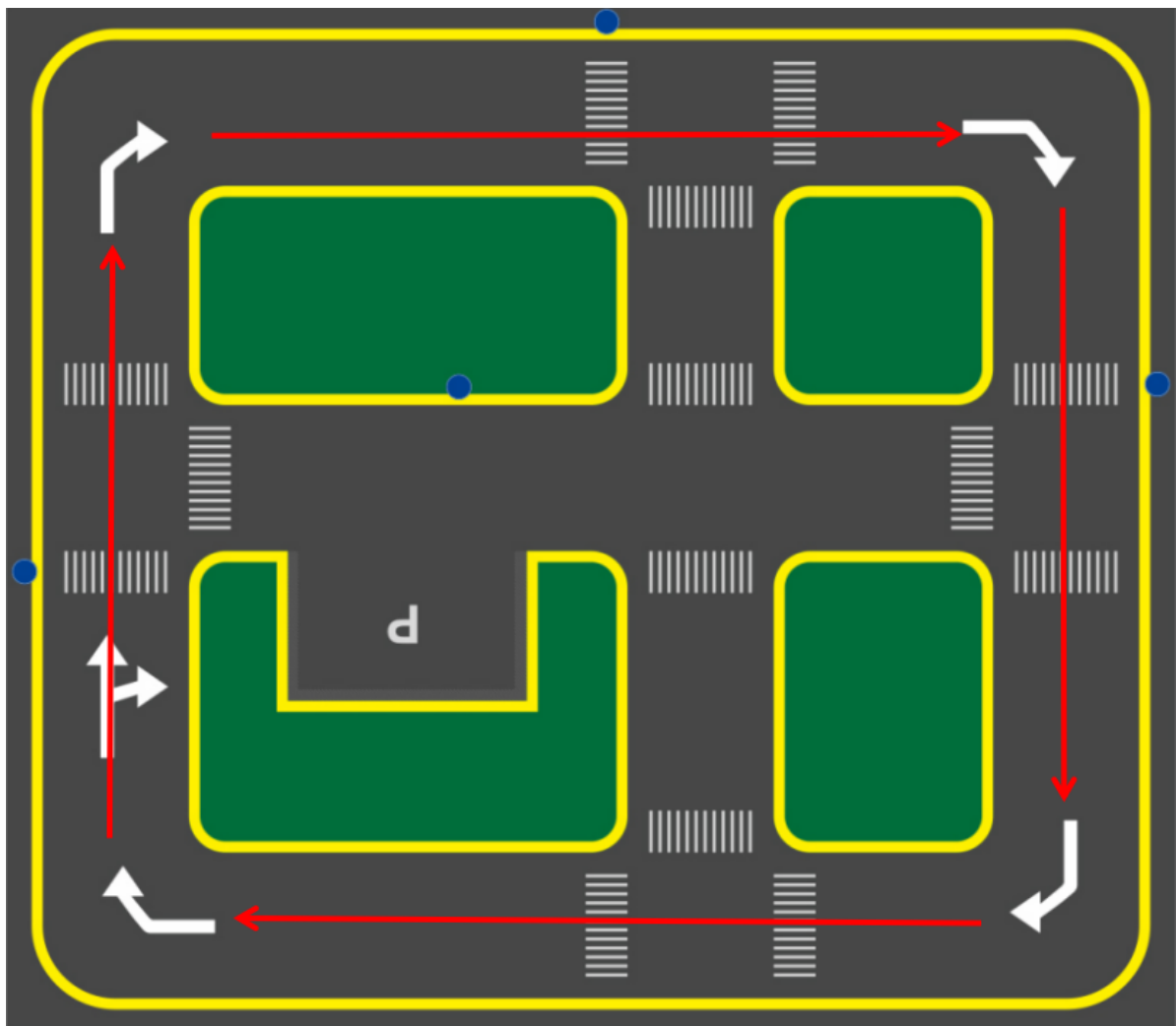
- Click AutoDrive to enter the parameter control panel. If there is no `AutoDrive` on the left side when you first open it, click `Refresh` below



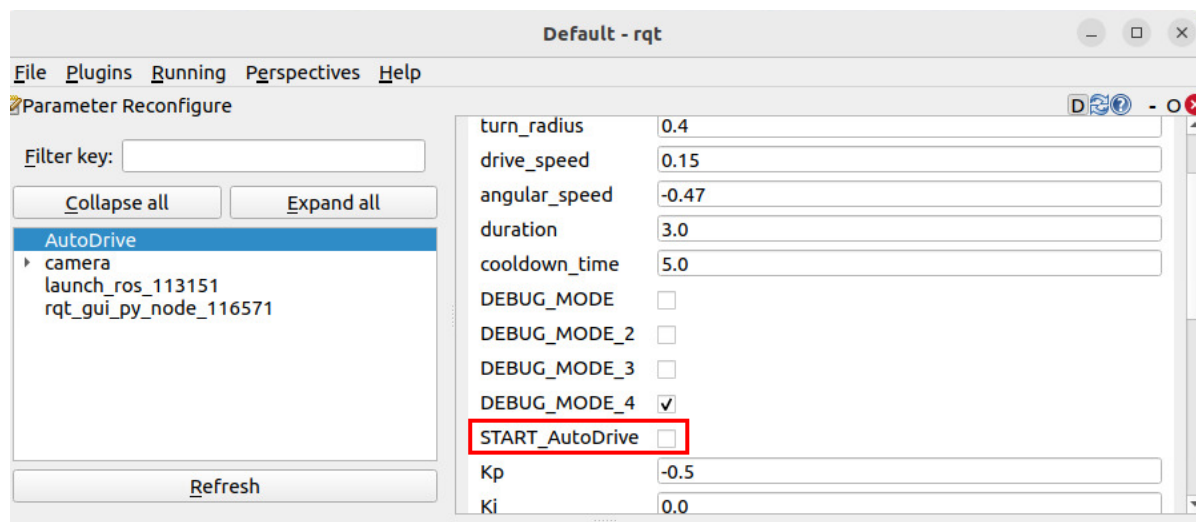
- Click `START_AutoDrive` to start autonomous driving and lane-keeping functionality



- After that, the vehicle will perform autonomous driving along the outer lane
 - In straight road sections, it will automatically execute straight driving and left/right lane centering
 - In curved road sections, it will automatically turn and return to lane-keeping mode after exiting the curve



- If you need to stop, simply uncheck `START_AutoDrive`



(System message might be shown here when necessary)

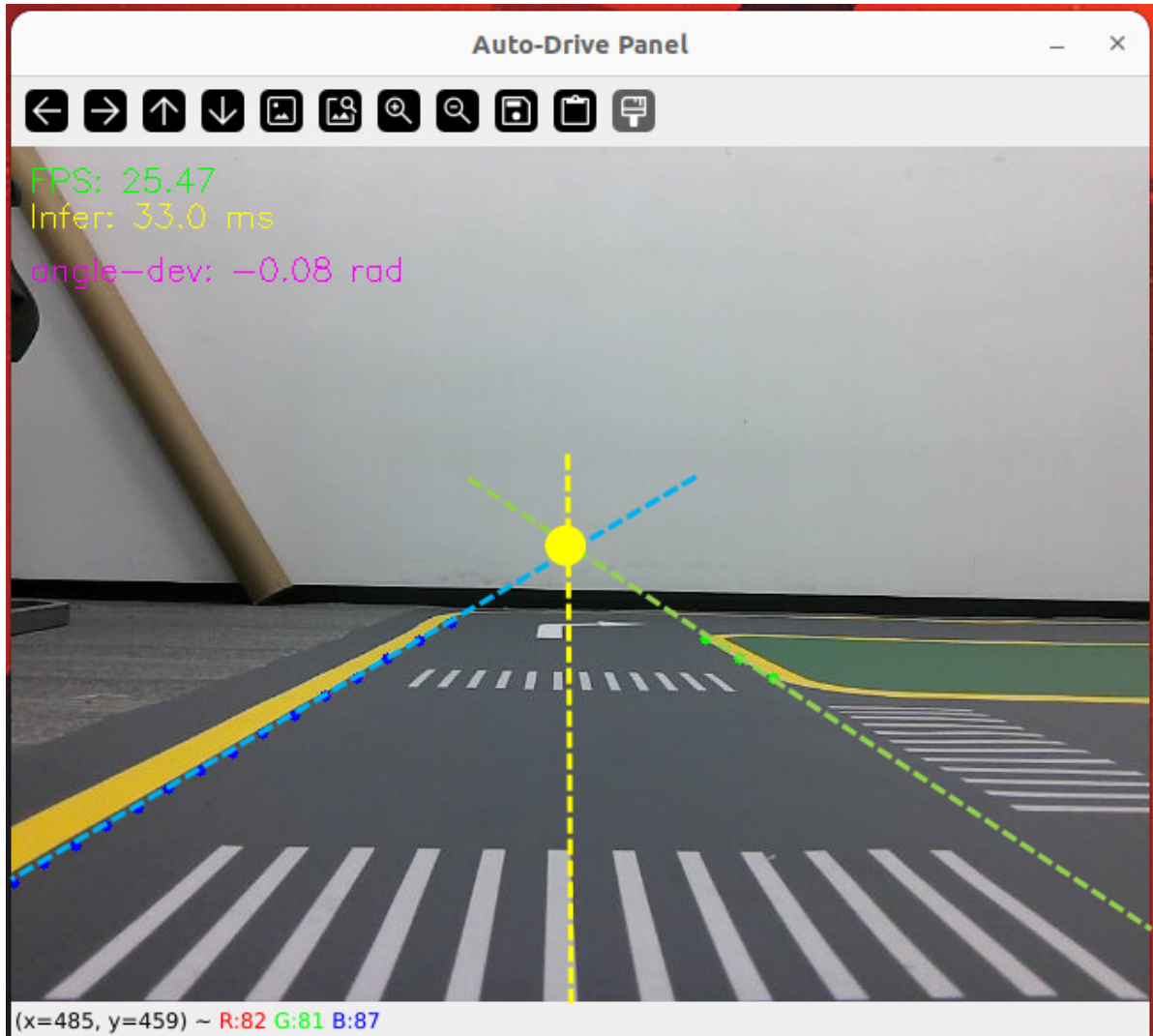
[!TIP]

If the lane-keeping autonomous driving effect does not meet expectations, refer to this chapter's [12-Autonomous Driving Debugging Special Topic](#) for parameter debugging.

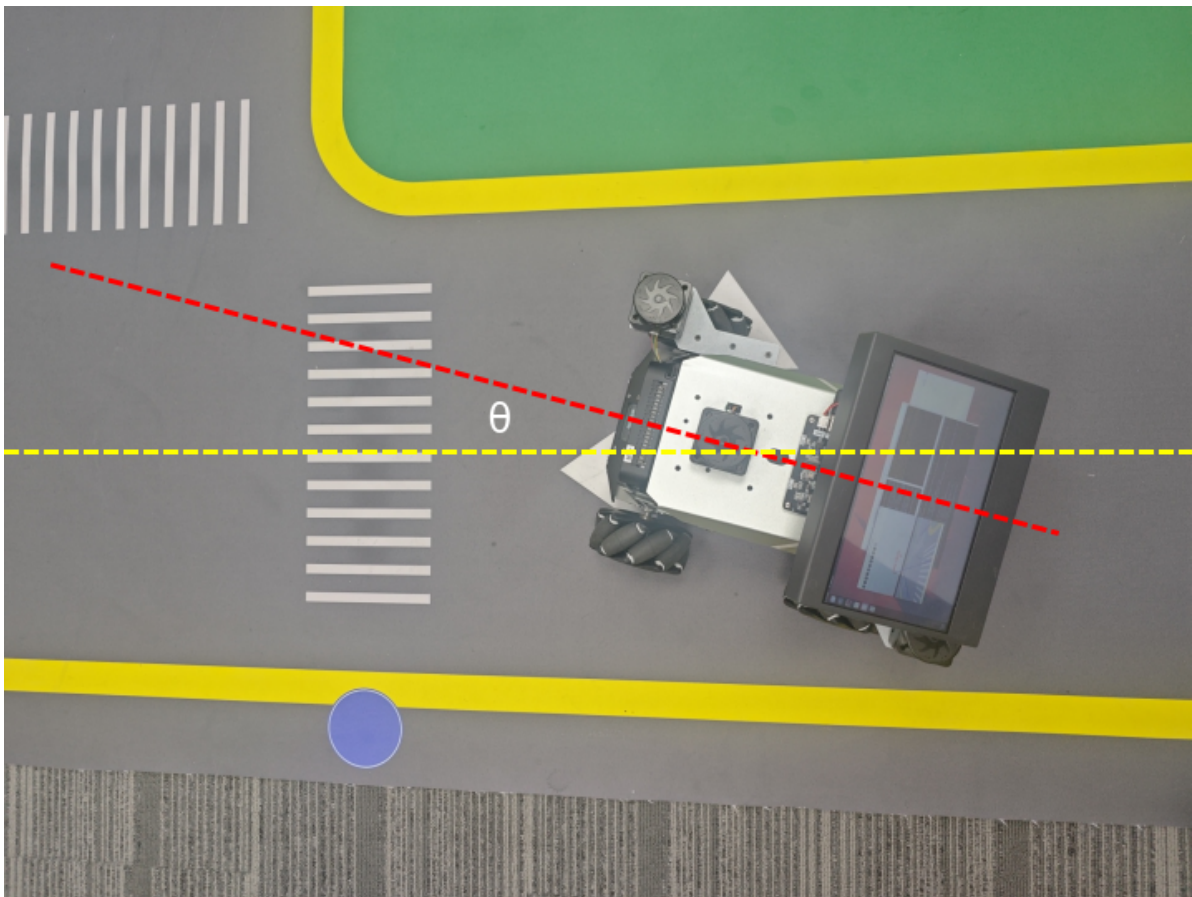
3. Principle Analysis

3.1 Straight Driving Maintenance

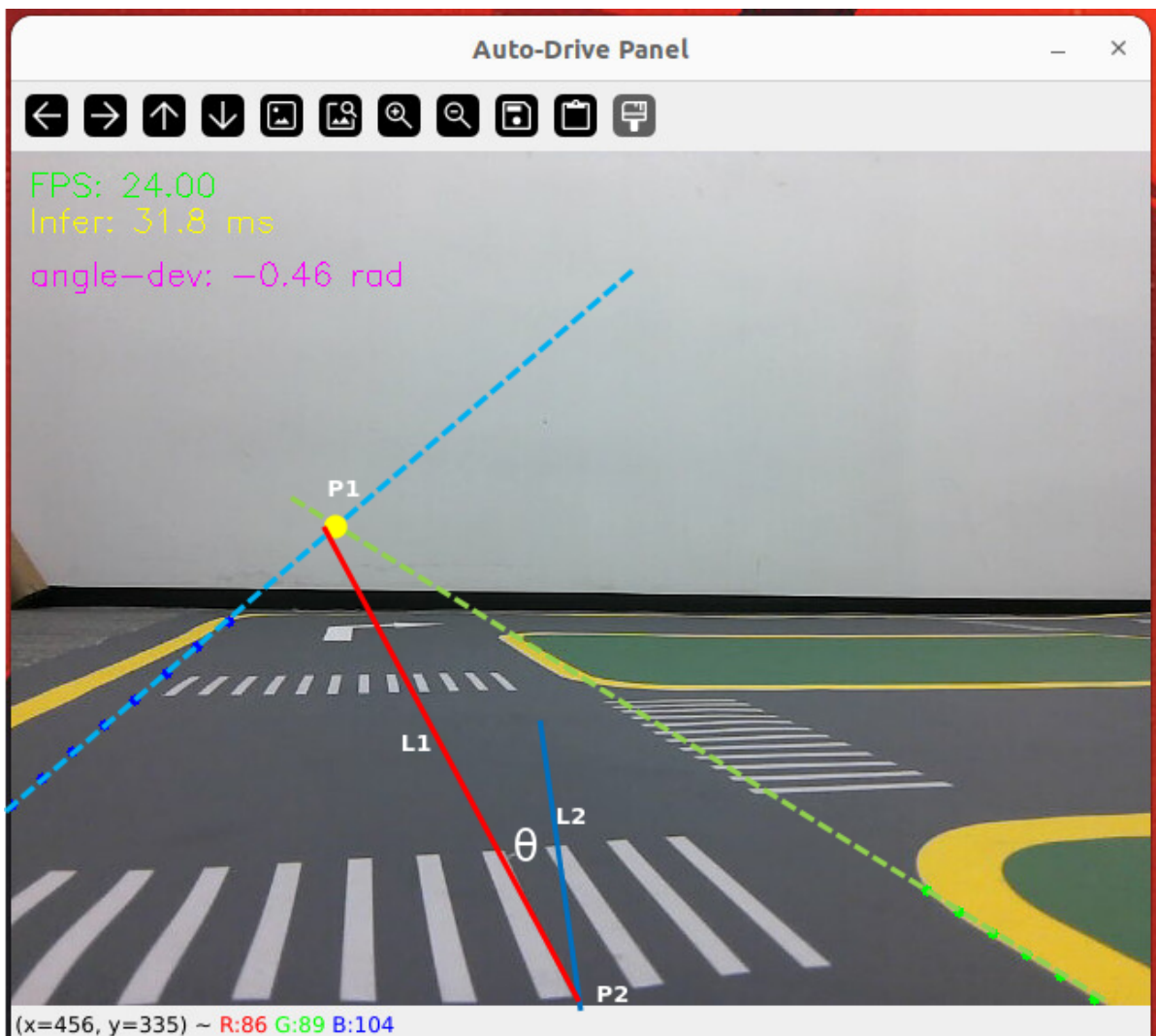
- According to perspective principles, in the camera perspective, the extensions of the left lane line, right lane line, and lane center line must intersect at a point (yellow dot in the figure), recorded as point P1. Using P1 can roughly estimate the position of the lane center line.



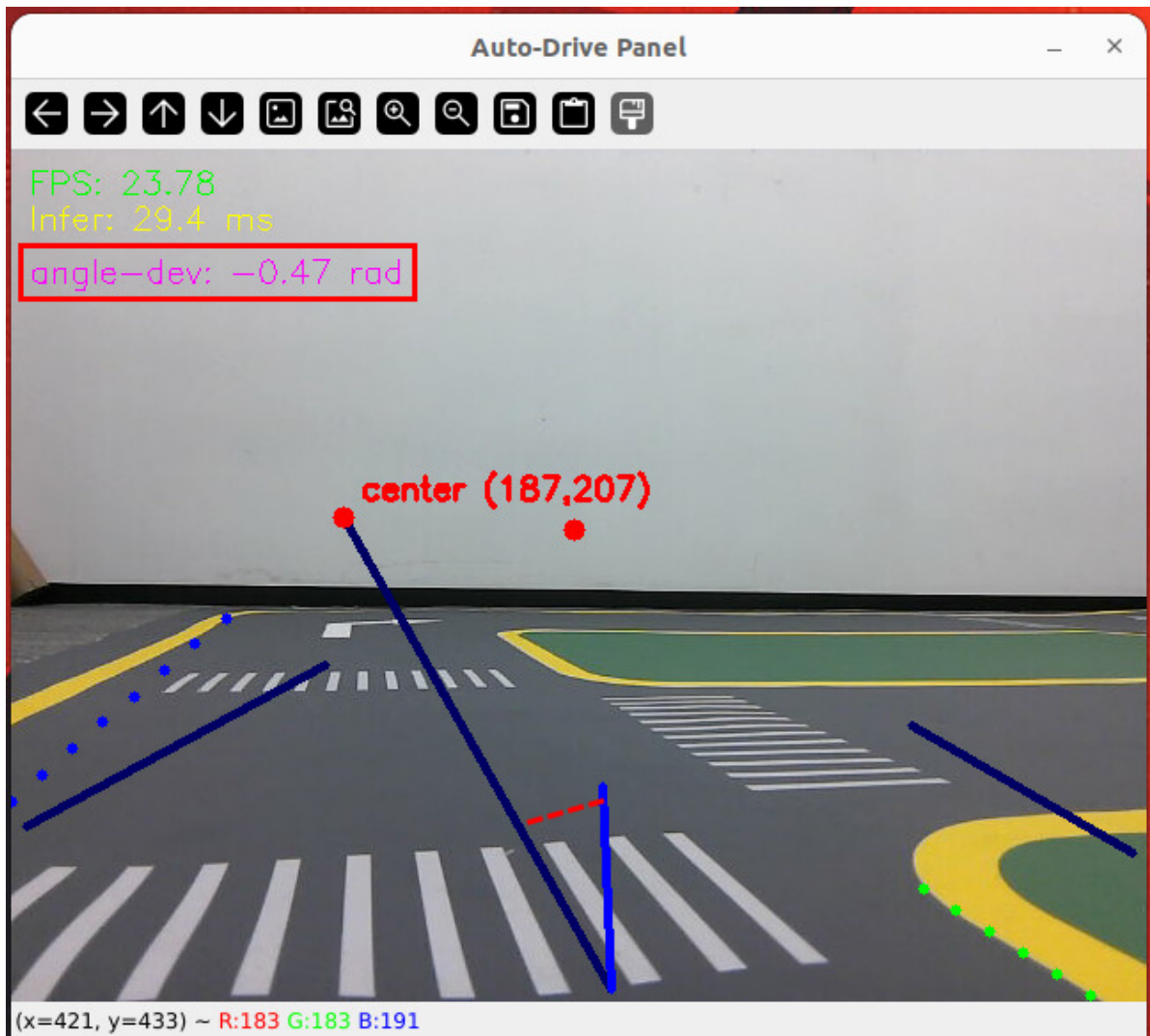
- Open `DEBUG_MODE_4` in the dynamic parameter configuration panel to display the connection between the centerline starting point and the lane line intersection point
 - The center dot is the intersection point P1 of the left and right lane lines (blue dotted line and green dotted line)
 - The green dot in the figure is the starting point of the vehicle body centerline, recorded as P2
 - The vehicle body centerline is recorded as L2, and the connection between the centerline starting point and the lane line intersection point is recorded as L1
- When the vehicle body tilts to one side, the two line segments L1 and L2 in the image will show a deviation angle



- At this time, the lane straight maintenance problem can be abstracted to finding the deviation angle problem



- For example, when the vehicle head tilts to the right and the deviation angle exceeds a certain threshold, PID control is triggered to correct the vehicle head direction.



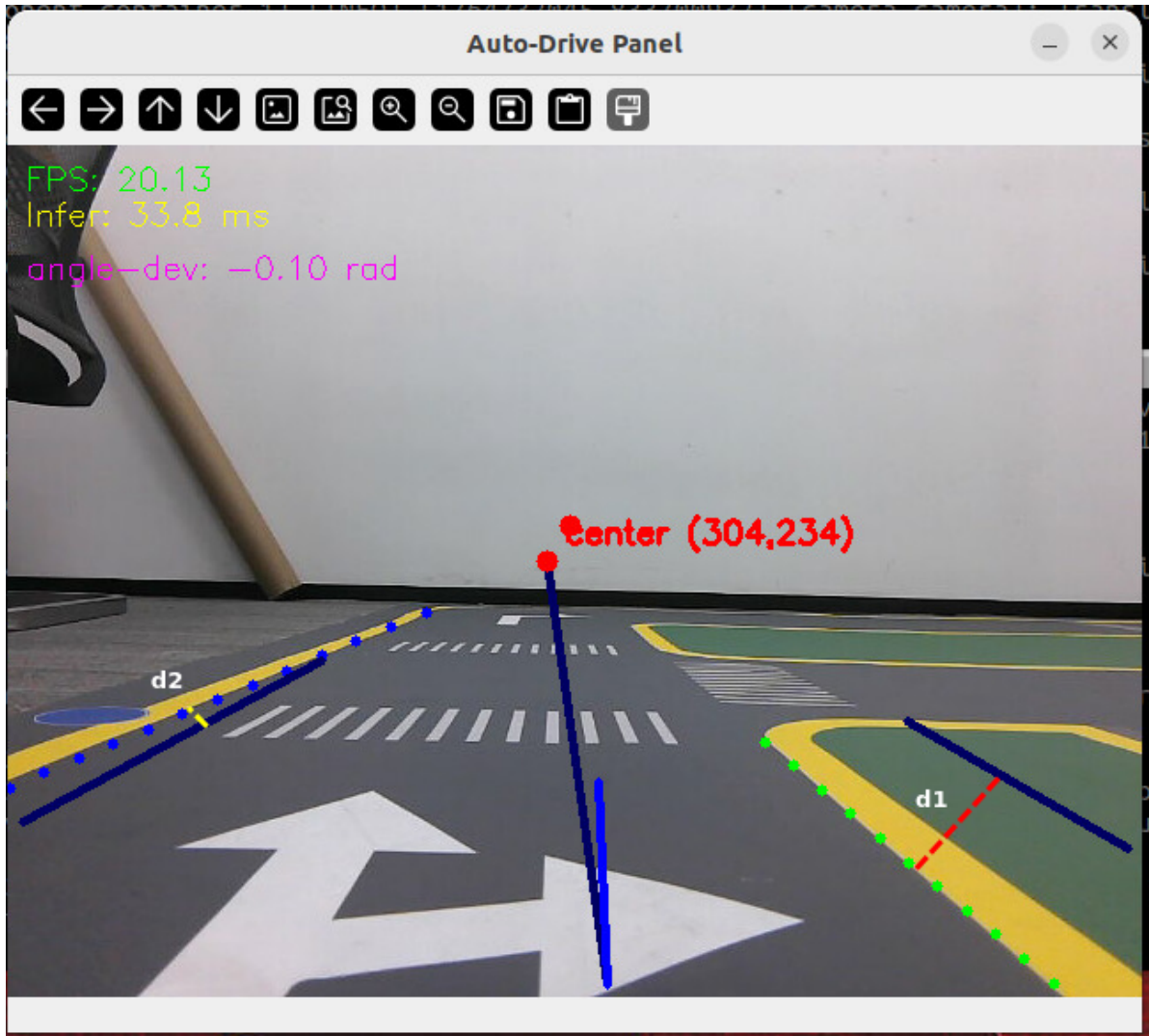
3.2 Left and Right Lane Keeping

- When the vehicle body deviates left and right from the lane centerline, according to the previously calibrated standard lane line position, the left and right deviation can be calculated. Through PID regulation, the vehicle body can be stabilized near the center position.

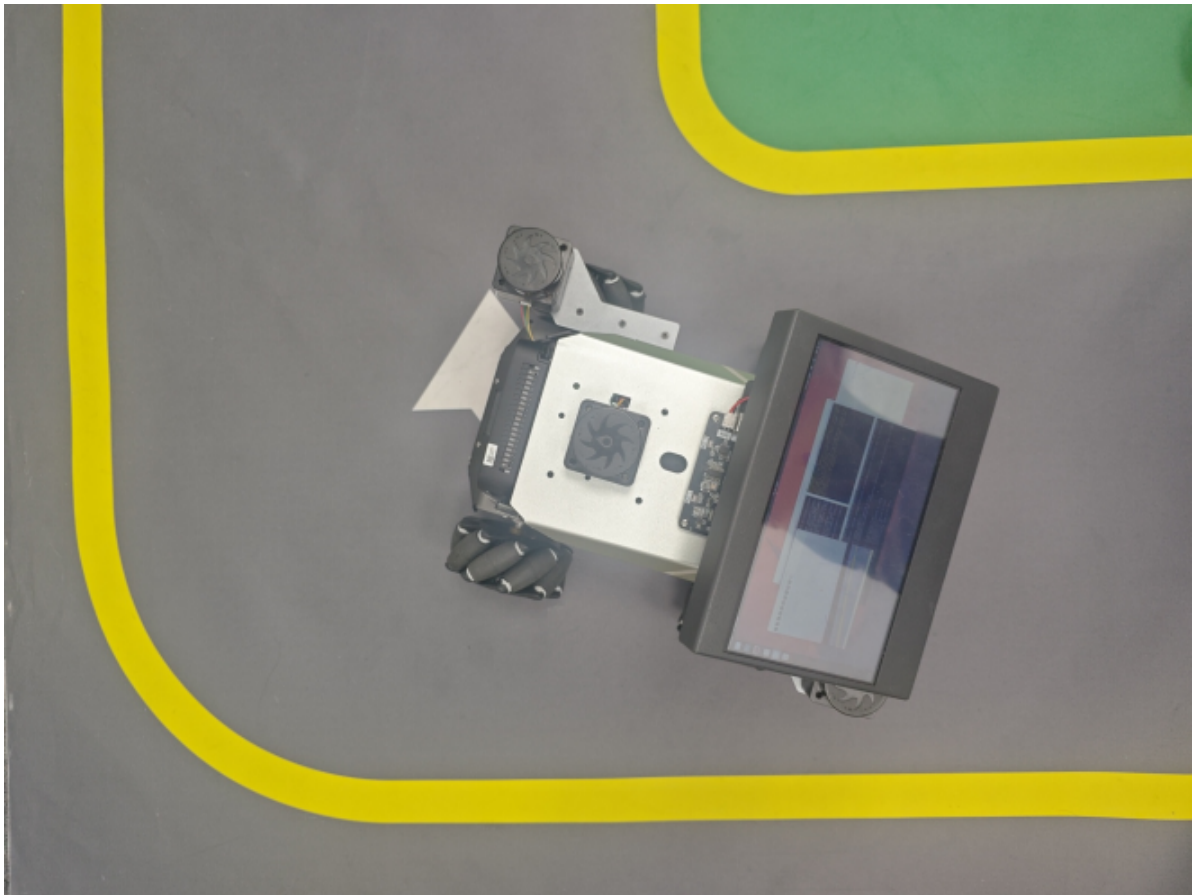
[!IMPORTANT]

- Under ideal conditions, the vehicle body drives along the lane center, and the real-time lane lines will fit the calibrated standard lane lines (left and right dark blue line segments). In actual driving, there will be an angle between the standard lane lines and the real-time lane lines, so the deviation of the vehicle body relative to the lane centerline can be calculated by calculating the distance between the real-time lane line midpoint and the standard lane line, where:
 - d_2 is the point-to-line segment distance between the left lane line midpoint and the calibrated lane line
 - d_1 is the point-to-line segment distance between the right lane line midpoint and the calibrated lane line
- Therefore, the deviation distance error of the vehicle body relative to the lane centerline can be equivalent to: $(d_2 - d_1) / 2$

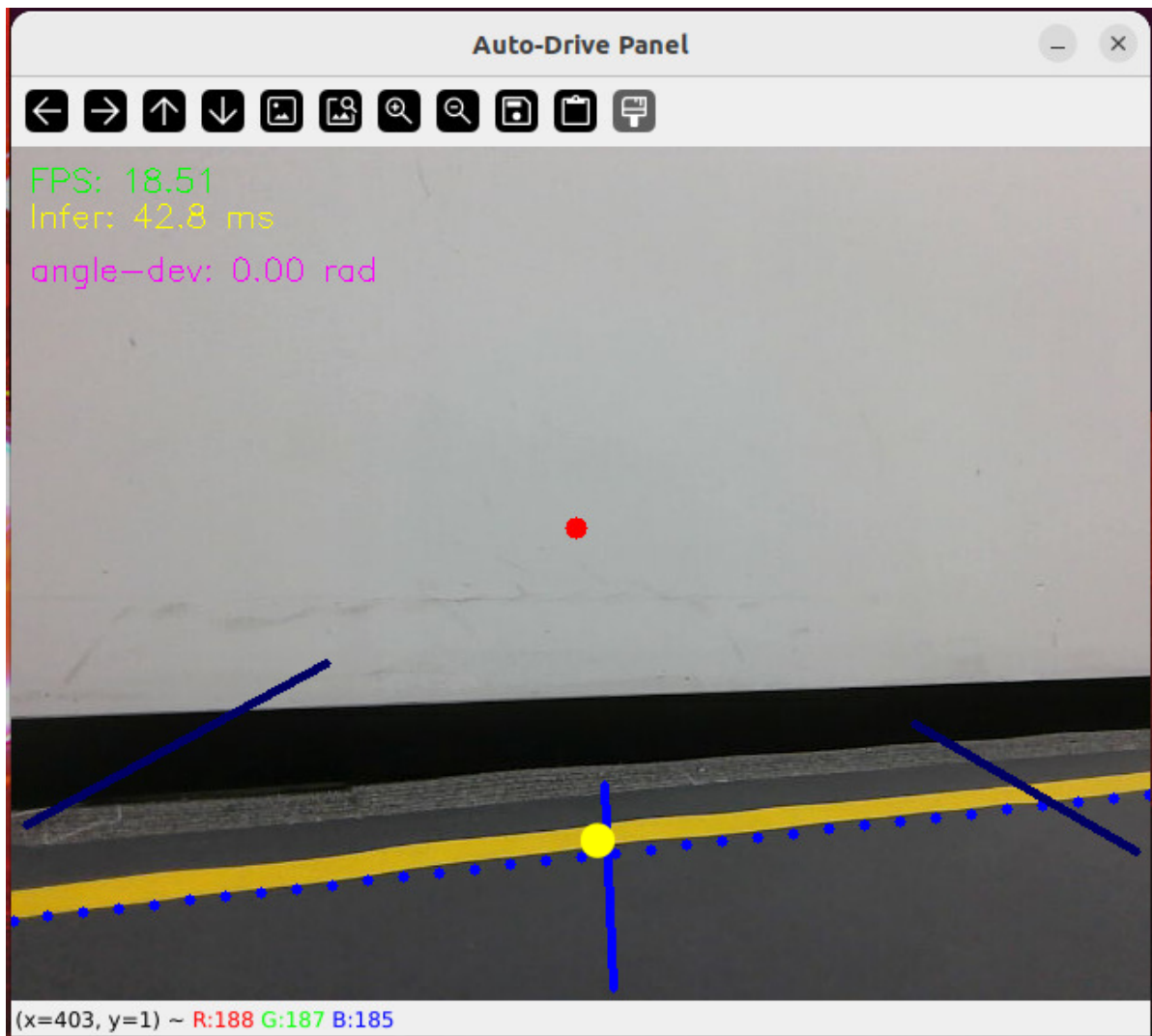
- $d1$ and $d2$ have positive and negative values. When the real-time lane line is on the left side of the calibrated lane line, it is negative.



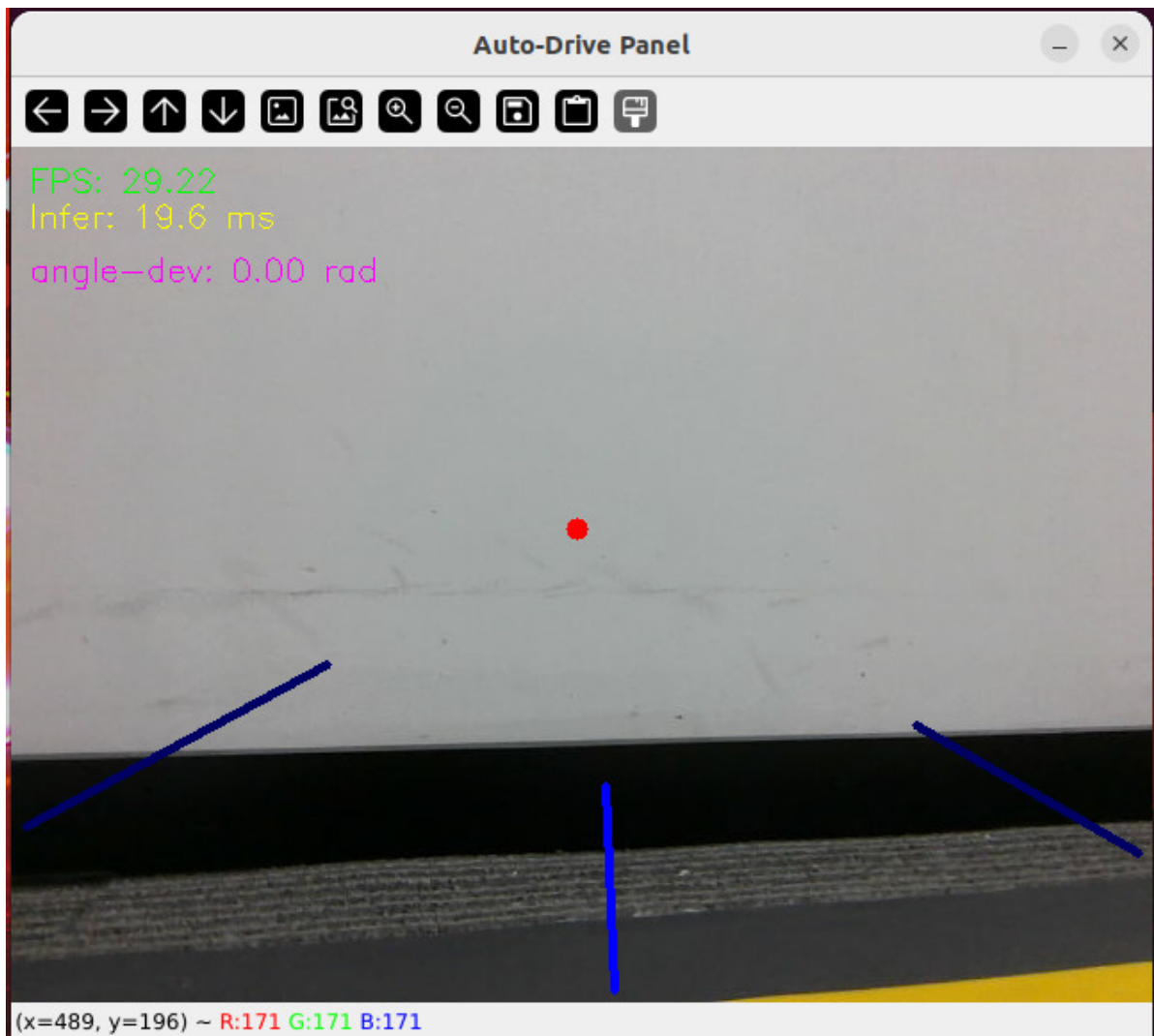
3.3 Curve Turning



- When the vehicle body enters a curve for turning, the vehicle body centerline will intersect with the left lane line in the camera perspective. Therefore, when there is an intersection point between the two, it proves that the vehicle body is close to the left lane line, and angular velocity control is used to turn the vehicle body.
- Therefore, **the length of the vehicle body centerline determines the timing of the vehicle entering the curve for turning.**



- Another situation is that due to YOLO not detecting lane lines or camera field of view blind spots, no lane lines can be seen. In this case, the vehicle will maintain the turning state from the previous frame and perform in-place rotation at a slow angular velocity until lane lines appear in the image.



4. Source Code Analysis

The logic source code of the program is located in the `auto_drive.py` program of the `auto_drive` package

```
~/yahboomM3_ws/code_ws/src/auto_drive/auto_drive/auto_drive.py
```

- The overall implementation program for processing lane lines is the `handle_lane` method in the `AutoDrive` class
 - Processing flow: Continuously obtain video frames from the `image_queue` queue, perform YOLO lane line detection to calculate vehicle body angle deviation and left-right deviation, then output speed through PID calculation to control vehicle body posture.

```
def handle_lane(self)->None:
    '''Processing lane line recognition detection results'''

    while True:
        try:
```

```

        frame = self.lane_detect.image_queue.get(timeout=0.1)
    except queue.Empty:
        continue
    infer_start = time.time()
    results = self.lane_detect.get_infer_result(frame, stream=True)
    #Initialize default values
    left_box = None
    right_box = None
    left_candidates = []
    right_candidates = []

    for res in results:
        boxes = res.boxes # Boxes object for bounding box outputs
        if self.DEBUG_MODE: frame = res.plot()
        if boxes is not None:
            for i in range(len(boxes)):
                box_coords = boxes.xyxy[i].tolist() # Get the
                # bounding box coordinates (xyxy format: xmin, ymin, xmax, ymax)

        lane=self.lane_detect.classify_lane_direction(frame,box_coords,debug=self.DEBUG
        _MODE)

        if lane=="left":
            left_candidates.append(box_coords)
        elif lane=="right":
            right_candidates.append(box_coords)

        if len(left_candidates)>0 :
            left_box = max(left_candidates, key=lambda box: box[3]) # select the
            # bounding box closest to the bottom (with the largest ymax).
        if len(right_candidates)>0:
            right_box = max(right_candidates, key=lambda box: ((box[2] -
            box[0])**2 + (box[3] - box[1])**2)**0.5)

        frame=self.__handle_lane_detect_result(left_box,right_box,frame)
        infer_end = time.time()

        self.frame_count_1 += 1
        curr_time = time.time()
        time_diff = curr_time - self.prev_time_1
        if time_diff >= 1.0:
            self.fps_1 = self.frame_count_1 / time_diff
            self.frame_count_1 = 0
            self.prev_time_1 = curr_time
            cv2.putText(frame,f"FPS: {self.fps_1:.2f}",(10,
            25),cv2.FONT_HERSHEY_SIMPLEX,0.6,(0, 255, 0),1,)

            infer_time = (infer_end - infer_start) * 1000
            cv2.putText(frame, f"Infer: {infer_time:.1f} ms", (10,
            45),cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0, 255, 255), 1)
            cv2.imshow("Auto-Drive Panel", frame)
            cv2.waitKey(1)

```

- The `__handle_lane_detect_result` method in the `AutoDrive` class is the main logic for implementing lane keeping

```

def __handle_lane_detect_result(self, left_box, right_box, frame):
    '''Lane centering processing'''
    error_mode = 99
    angular_z=0.0
    error_angular = [0.0, 0.0]

    if left_box is not None: k_left=self.lane_detect.draw_left_line(frame,
left_box)
    if right_box is not None:
k_right=self.lane_detect.draw_right_line(frame, right_box)

    if left_box is not None and right_box is not None and left_box[0]
<right_box[0] :
        error_mode=0

    frame,x_center,y_center=self.lane_detect.draw_center_point(frame,left_box,right
_box,debug=self.DEBUG_MODE_4) #Draw the real-time lane center

    avg_offset=self.lane_detect.cal_point_to_lane_line_distance(left_box,right_box)
    #Calculate the deviation of the car in the lane
    if self.DEBUG_MODE_3 : self.__calculate_init_error(avg_offset)
        #Initial error compensation

    error_angular=self.lane_detect.cal_error_angular(x_center,y_center)#Calculate
the angular error between the vehicle body and the center point

    elif left_box is not None and right_box is None:
        error_mode=1
        self.lane_detect.prev_center=None
    elif right_box is not None and left_box is None :
        error_mode=2
        self.lane_detect.prev_center=None
    elif left_box is None and right_box is None:
        error_mode=3
        self.lane_detect.prev_center=None

    if self.lanekeeping_enable:
        '''Lane keeping mode'''
        if error_mode==0 :
            '''Both the left and right lane lines exist'''
            if abs(error_angular[1])> self.min_error_angular:
                angular_z=self.pid_controller.pid_control(error_angular[1])
                angular_z=min(angular_z,0.3)
                angular_z=max(angular_z,-0.3)

self.__set_current_speed(linear_x=self.drive_speed,angular_z=angular_z)
        else:
            linear_y=self.pid_controller_1.pid_control(avg_offset-10.18)
            linear_y=min(linear_y,0.05)
            linear_y=max(linear_y,-0.05)

self.__set_current_speed(linear_x=self.drive_speed,linear_y=linear_y)

    elif error_mode==1:
        '''only the left lane line'''
        if_infer=self.lane_detect.IF_lane_intersection(left_box)
        if if_infer is not None:

```

```

        # If infer_result is true, it means there is an intersection
        point, and control is then performed based on the angle deviation.
        angular_z=self.angular_speed
    else:
        if left_box[0]>self.lane_detect.start_point[0]:
            angular_z=self.angular_speed
        else:
            angular_z=0.0

self.__set_current_speed(linear_x=self.drive_speed,angular_z=angular_z)

    elif error_mode==2:
        '''Only the right lane line'''
        if right_box[0]<self.lane_detect.start_point[0]:
            self.turn_right_conunt+=1
            if self.turn_right_conunt>self.tr_threshold:
                angular_z=-self.angular_speed

self.__set_current_speed(linear_x=0.1,angular_z=angular_z)
        self.turn_right_conunt=0
    else:
        self.turn_right_conunt=0
    elif error_mode==3:
        '''Neither the left nor the right lane lines exist'''
        if self.last_error_mode==1:
            '''If you are in a blind spot and cannot see the left lane
line in front of you, turn right'''
            self.__set_current_speed(linear_x=0.0,angular_z=-0.15)
        elif self.last_error_mode==2:
            '''If you are in a blind spot and cannot see the lane line
on the right in front of you, turn left'''
            self.__set_current_speed(linear_x=0.0,angular_z=0.09)
        elif self.last_error_mode==0:
            self.__set_current_speed(linear_x=0.1)
    else:
        '''The rest of the situation'''

self.__set_current_speed(linear_x=self.drive_speed,angular_z=0.0)

    self.last_error_mode=error_mode

    # Display angular deviation on the image
    cv2.putText(frame, f"angle-dev: {error_angular[1]:.2f} rad", (10, 75),
cv2.FONT_HERSHEY_SIMPLEX, 0.6, (255, 0, 255), 1)
    if self.DEBUG_MODE_4: self.lane_detect.draw_midline(frame)
    return frame

```

- The program for calculating deviation angle and left-right deviation is as follows:

```

        if left_box is not None and right_box is not None and left_box[0]
        <right_box[0] :
            error_mode=0

    frame,x_center,y_center=self.lane_detect.draw_center_point(frame,left_box,right
    _box,debug=self.DEBUG_MODE_4) #Draw the real-time lane center

    avg_offset=self.lane_detect.cal_point_to_lane_line_distance(left_box,right_box)
    #Calculate the deviation of the car in the lane
    if self.DEBUG_MODE_3 : self.__calculate_init_error(avg_offset)
    #Initial error compensation

    error_angular=self.lane_detect.cal_error_angular(x_center,y_center)#Calculate
    the angular error between the vehicle body and the center point

```

- The calculation steps `draw_center_point`, `cal_point_to_lane_line_distance`, `__calculate_init_error`, `cal_error_angular` belong to the `Lane_Detect` class in `yolov11_infer.py`
- `draw_center_point` is used to draw the connection between the real-time lane center point and the vehicle body centerline starting point

```

def draw_center_point(self,image,left_box,right_box,debug=True):
    '''Draw the center point of the lane and connect it with the starting
    point of the central axis of the vehicle's perspective'''
    x1l, y1l, x2l, y2l = map(int, left_box)
    x1r, y1r, x2r, y2r = map(int, right_box)
    left_p1 = (x1l, y2l)
    left_p2 = (x2l, y1l)
    right_p1 = (x1r, y1r)
    right_p2 = (x2r, y2r)

    k1, b1 = self._line_params(left_p1, left_p2)
    k2, b2 = self._line_params(right_p1, right_p2)

    # Calculate the intersection point
    if abs(k1 - k2) < 1e-6:
        x_center, y_center = image.shape[1] // 2, image.shape[0] // 2
    else:
        x_center = int((b2 - b1) / (k1 - k2))
        y_center = int(k1 * x_center + b1)
        x_center_smooth,
    y_center_smooth=self.center_point_filter(x_center,y_center)
        if debug: cv2.line(image, (x_center_smooth,y_center_smooth),
    self.start_point, (100, 0, 0), 3) #The line connecting the center point of
    the road estimation and the starting point of the central axis of the vehicle's
    viewing Angle

        if debug:
            cv2.circle(image, (x_center_smooth, y_center_smooth), 6, (0, 0,
    255), -1)
            cv2.putText(image, f"center ({x_center_smooth},{y_center_smooth})",
                (x_center_smooth + 10, y_center_smooth - 10),

```

```
cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0, 0, 255), 2)
```

```
return image, x_center_smooth, y_center_smooth
```

- `cal_point_to_lane_line_distance` method is used to calculate the left and right deviation of the vehicle body in the lane

```
def cal_point_to_lane_line_distance(self, left_box, right_box):  
    '''  
    Calculate the left and right deviations of the vehicle body in the lane  
    '''  
    x1l, y1l, x2l, y2l = map(int, left_box)  
    x1r, y1r, x2r, y2r = map(int, right_box)  
  
    left_p1 = (x1l, y1l)  
    left_p2 = (x2l, y1l)  
  
    right_p1 = (x1r, y1r)  
    right_p2 = (x2r, y2r)  
  
    left_point = np.array(self.left_lane_center_point)  
    right_point = np.array(self.right_lane_center_point)  
  
    left_distance =  
self._point_to_infinite_line_distance_with_direction(left_point, left_p1,  
left_p2)  
    right_distance =  
self._point_to_infinite_line_distance_with_direction(right_point, right_p1,  
right_p2)  
    avg_offset = (left_distance - right_distance) / 2  
    return avg_offset
```

- Curve turning program implementation

In the previous `__handle_lane_detect_result` method, the corresponding program segment is as follows. If the vehicle body centerline intersects with the left lane line, the vehicle body turns at `self.angular_speed` angular velocity.

```

elif error_mode==1:
    '''only the left lane line'''
    if_infer=self.lane_detect.IF_lane_intersection(left_box)
    if if_infer is not None:
        # If infer_result is true, it means there is an intersection
        point, and control is then performed based on the angle deviation.
        angular_z=self.angular_speed
    else:
        if left_box[0]>self.lane_detect.start_point[0]:
            angular_z=self.angular_speed
        else:
            angular_z=0.0

self.__set_current_speed(linear_x=self.drive_speed,angular_z=angular_z)

```

```

def IF_lane_intersection(self, left_box):
    """
    Determine whether the left lane line intersects the vehicle's central
    axis and return to the intersection point
    """
    x1l, y1l, x2l, y2l = map(int, left_box)

    left_p1 = (x1l, y2l)
    left_p2 = (x2l, y1l)
    x1, y1 = self.start_point
    x2, y2 = self.end_point
    x3, y3 = left_p1
    x4, y4 = left_p2

    denom = (x1 - x2)*(y3 - y4) - (y1 - y2)*(x3 - x4)
    if denom == 0:
        return None

    t = ((x1 - x3)*(y3 - y4) - (y1 - y3)*(x3 - x4)) / denom
    u = ((x1 - x3)*(y1 - y2) - (y1 - y3)*(x1 - x2)) / denom

    if 0 <= t <= 1 and 0 <= u <= 1:
        inter_x = x1 + t * (x2 - x1)
        inter_y = y1 + t * (y2 - y1)
        return (inter_x, inter_y)
    else:
        return None

```

- When lane lines cannot be detected, trigger self-rescue mechanism and slowly rotate to find lane lines based on the previous turning state

```

elif error_mode==3:
    '''Neither the left nor the right lane lines exist'''
    if self.last_error_mode==1:
        '''If you are in a blind spot and cannot see the left lane
line in front of you, turn right'''
        self.__set_current_speed(linear_x=0.0,angular_z=-0.15)
    # elif self.last_error_mode==2:
    #     '''If you are in a blind spot and cannot see the lane line
on the right in front of you, turn left'''
    #     self.__set_current_speed(linear_x=0.0,angular_z=0.09)
    elif self.last_error_mode==0:
        self.__set_current_speed(linear_x=0.1)
    else:
        '''The rest of the situation'''

self.__set_current_speed(linear_x=self.drive_speed,angular_z=0.0)

```